

PREPSPHERE: DESIGN AND DEVELOPMENT OF A FULL-STACK PLACEMENT RESOURCE SHARING AND PEER MENTORSHIP PLATFORM

A PROJECT REPORT

submitted by

RAGUL A (230701252)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



**RAJALAKSHMI ENGINEERING COLLEGE,
ANNA UNIVERSITY CHENNAI 602 105**

MAY 2026

BONAFIDE CERTIFICATE

Certified that this project report titled **“PREPSPHERE: DESIGN AND DEVELOPMENT OF A FULL-STACK PLACEMENT RESOURCE SHARING AND PEER MENTORSHIP PLATFORM”** is the Bonafide work of RAGUL A (230701252), who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Student Signature with Name

RAGUL A

Signature of the Supervisor with date

Signature Examiner-1

Signature Examiner-2

ANNEXURE

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
1.	Introduction -----	4
1.1	Agile Development approach -----	4
1.2	Full - Stack Engineering Methodology -----	8
2.	Literature review -----	12
2.1	Domain-Specific Research -----	12
2.2	Software Engineering Research-----	17
3.	Domain area -----	22
3.1	Key Aspects of Domain -----	22
3.2	Challenges specific to the project -----	23
4.	Requirements Analysis Stage -----	25
4.1	Activities -----	25
4.2	Primary Research -----	26
4.3	Secondary Research -----	32
4.4	Outcome of the empathize phase -----	33
5.	System Design Stage -----	36
5.1	User Needs Identified -----	36
5.2	Brainstorming to Define Problem Statements -----	36
5.3	Selected Final Architecture -----	37
6.	Ideation & Future Planning Stage -----	38
6.1	Analysis of System Requirements-----	38
6.2	Feature Mind Mapping-----	38
6.3	Results of Planning Session-----	38
6.4	Selected Best Approach -----	39
6.5	Value Proposition Statement-----	39
7.	Prototype & Development Stage -----	40
7.1	Details of the Prototype-----	40
7.2	Prototype Visuals-----	40
8.	Test and feedback -----	42
8.1	Test-----	42

8.2 Feedback.....	42
9. Re-design and implementation	43
9.1 Re-Design Actions Based on Feedback	43
9.2 Implementation.....	44
10. Conclusion.....	45
11. Future work.....	46
12. Learning outcome of Software Engineering Methodology.....	47
13. References	48
13.1 Journals.....	48
13.2 Software Engineering Papers.....	48

1 INTRODUCTION

The placement preparation landscape for engineering students is fragmented, inefficient, and inequitable. Seniors who have successfully navigated campus placements accumulate a wealth of company-specific resources, aptitude materials, coding guides, and experiential knowledge — yet this invaluable information rarely reaches juniors in a structured, accessible format. The result is a recurring cycle where each batch of students reinvents the wheel, wastes time sourcing unreliable materials, and misses the mentorship that could dramatically improve their outcomes.

Embracing the principles of Agile Software Development and Full-Stack Engineering, we sought to deeply understand these challenges from the end-user's perspective and co-create a digital solution that is practical, scalable, and transformative. Our platform, PrepSphere, was conceptualized and developed through iterative sprints — Requirements Analysis, System Design, Ideation, Prototyping, Development, and Testing — grounded in real feedback from both senior and junior students.

The resulting platform combines JWT-secured role-based authentication, a resource feed with LazyColumn-style browsing, file upload (PDF/image), a like/bookmark/comment interaction system, tag-based filtering (DSA, Aptitude, Company-wise), real-time notifications, and a peer chat mechanism. User testing across student cohorts demonstrated a measurable improvement in resource discovery time and a significant increase in placement preparation engagement. PrepSphere sets a new standard for peer-driven, technology-enabled placement mentorship. By transforming informal, scattered mentorship into a structured digital ecosystem, PrepSphere empowers every junior student — regardless of their social network or college connections — to access the same quality of placement guidance that was previously available only to a privileged few.

1. AGILE DEVELOPMENT APPROACH

The Agile Development approach guided the end-to-end engineering of PrepSphere, keeping user needs, iterative feedback, and working software at the center of every decision. Recognizing that placement preparation is a deeply personal, time-sensitive journey, our team adopted a build-measure-learn mindset that prioritizes working features, user collaboration, and continuous improvement.

1. REQUIREMENTS ANALYSIS (EMPATHIZE EQUIVALENT):

- ♦ In the Requirements Analysis phase, we engaged directly with senior students (resource providers) and junior students (resource consumers) through surveys, interviews, and focus groups. We uncovered critical pain points: juniors struggled to find company-specific materials; seniors had no structured way to share resources; there was no search, tagging, or mentorship mechanism in existing tools.
- ♦ This phase revealed technical, emotional, and logistical friction. For example, juniors frequently downloaded outdated PDFs from random drives, and seniors had no visibility into whether their shared materials were helpful. These insights formed the foundation for PrepSphere's feature set. Students with better senior connections consistently outperformed peers in placements — not due to aptitude, but due to privileged access to resources.

2. SYSTEM DESIGN:

- ♦ In the System Design phase, we synthesized research insights into a clear technical problem statement and architecture. Three recurring pain points emerged: the lack of a centralized, searchable resource repository; the absence of role-based access for seniors and juniors; and the non-existence of interactive features like likes, bookmarks, and comments.

- These challenges were framed into a central problem statement: "How might we design an inclusive, technology-driven resource sharing platform that ensures timely response, accessible to queries, and reliable interactivity for users across varying literacy levels?"
- This definition provided focused direction for feature prioritization, ensuring PrepSphere would directly address real-world friction in placement preparation workflows.

3. IDEATION & FEATURE PLANNING:

- During the Ideation phase, we translated research insights into concrete features. Planning sessions among the development team led to the conceptualization of PrepSphere — a full-stack placement resource platform integrating resource upload, feed browsing, search and filtering, interaction (like/comment/bookmark), tagging (DSA, Aptitude, Company-wise), notifications, and peer chat.
- Innovative engineering decisions included JWT-based role differentiation, Multer-powered file uploads stored server-side, Provider-based state management in Flutter, and a MongoDB schema supporting nested likes and comments arrays.

4. PROTOTYPE & DEVELOPMENT:

- In the Prototype and Development stage, we built a functional full-stack application. The Flutter frontend used Riverpod/Provider for state management, Dio for API calls, and GoRouter for navigation. The Node.js + Express backend exposed RESTful APIs secured by JWT middleware. MongoDB collections stored Users, Resources, Comments, Likes, and Notifications.

- The prototype demonstrated the complete user flow — signup with role selection, resource upload by seniors, feed browsing by juniors, search/filter, and like/comment interactions — validating that an integrated, scalable placement platform could be built with affordable, widely-adopted technologies.

5. TESTING & FEEDBACK:

- Testing was conducted through pilot sessions with senior and junior student cohorts. Users interacted with PrepSphere under guided and unguided conditions while the team recorded usability issues, API errors, and qualitative feedback. Results showed a significant improvement in resource discovery speed and high satisfaction with the tag-based filtering system. Areas for improvement included refining the chat UI and adding offline caching for low-connectivity scenarios.

6. IMPLEMENT AND ITERATION:

- Following successful testing, PrepSphere was refined based on user feedback — improving API response handling, enhancing the resource card UI, and adding prescription-style validation for file uploads. The modular MVVM architecture ensured smooth scalability. PrepSphere is not a one-time release; it evolves with user needs, technology updates, and institutional requirements, making it a sustainable peer mentorship platform.

1.2 FULL-STACK ENGINEERING METHODOLOGY AND ITS PHASES

The Full-Stack Engineering methodology underpins PrepSphere's development, emphasizing iterative sprints, clean architecture, and user-centered feature delivery. We adopted this methodology to ensure PrepSphere would effectively solve real placement preparation challenges while remaining adaptable to diverse

student cohorts and institutional contexts. This approach fosters technical excellence and user-centric innovation.

1. REQUIREMENTS ANALYSIS:

We began by engaging directly with students across multiple engineering batches to understand their placement preparation habits, pain points, and expectations. Through surveys, interviews, and observation of existing informal sharing practices (WhatsApp groups, Google Drive links), we uncovered critical barriers: no structured tagging, no searchability, no role differentiation, and no interaction layer.

2. SYSTEM DESIGN:

Based on requirements analysis, we formulated a clear and concise design goal: "How can we create a full-stack mobile platform that delivers secure, role-based, and feature-rich placement resource sharing for senior and junior engineering students?". This guided architecture decisions: MVVM pattern, Flutter + Node.js + MongoDB stack, JWT authentication, and RESTful API design.

3. IDEATION:

With a well-defined problem, we conducted feature brainstorming involving student stakeholders and technology specialists. Ideas included tag-based resource categorization, real-time notifications, a chat system for peer mentorship, company-wise resource filters, and a bookmarking system. We evaluated concepts for feasibility, impact, and development effort, selecting the most valuable features for PrepSphere's initial release.

4. PROTOTYPE:

We developed a functional prototype of PrepSphere incorporating key features: role-based login, resource upload, feed browsing, and interaction. The Flutter UI used Material 3 design with a primary Indigo (#4F46E5) and secondary Cyan (#06B6D4) color palette. The backend was built with Node.js and Express, MongoDB managed data, and Multer handled file uploads. This allowed us to simulate end-to-end user interactions and validate core workflows before production deployment.

5. TESTING:

Testing involved deploying PrepSphere in real-world student settings — pilot sessions with engineering students preparing for campus placements. Users completed tasks such as uploading a resource, searching by tag, liking and commenting on a resource, and using the chat feature. Feedback was collected on ease of use, performance, and feature completeness. Testing confirmed high usability scores and identified optimization opportunities in notification delivery and chat message ordering.

2 LITERATURE REVIEW

The domain of placement preparation and peer mentorship in engineering institutions faces persistent challenges related to information asymmetry, resource discoverability, and structured knowledge transfer between student cohorts. Numerous studies and industry analyses highlight the critical need for integrated digital platforms that address the fragmented nature of informal resource sharing, improve access to company-specific preparation materials, and facilitate structured peer interaction. A growing body of work emphasizes the importance of role-based, user-centered platform design to create educational tools that are inclusive, searchable, and adaptable to diverse student populations

Research also underscores that scalable full-stack architectures combining secure authentication, file management, real-time interaction, and notification systems can significantly improve learning outcomes by reducing preparation time, improving resource quality access, and enabling mentorship at scale. This body of work informs the development of PrepSphere as a comprehensive, practical, and transformative peer mentorship platform.

2.1 DOMAIN-SPECIFIC RESEARCH

1. Role-Based Access Control in Educational Platforms: A Systematic Review

This study investigates the implementation of Role-Based Access Control (RBAC) in educational and collaborative platforms, highlighting the critical importance of differentiating user privileges based on roles such as mentor/senior and learner/junior. The research demonstrates that platforms with well-defined role hierarchies experience significantly higher engagement, lower misuse rates, and better content quality. The findings directly informed PrepSphere's senior-junior role architecture, where seniors have upload and mentorship privileges while juniors access, interact with, and bookmark shared resources.

The study concludes that RBAC, when combined with JWT-based stateless authentication, provides a scalable and secure foundation for peer learning platforms. It advocates for middleware-enforced role verification at the API layer — a pattern directly adopted in PrepSphere's Express.js backend.

2. Scalable File Upload Architectures for Mobile Applications

This paper presents approaches to managing file uploads (PDF, images, documents) in mobile-first full-stack applications, comparing server-side storage (Multer + local filesystem), cloud object storage (AWS S3, Firebase Storage), and hybrid approaches.

The research demonstrates that for early-stage platforms with moderate upload volumes, Multer-based server-side storage with URL reference saving in MongoDB provides the optimal balance of simplicity and performance. For scale, migration to object storage is recommended.

The proposed architecture — multipart form data from mobile client, Multer middleware on Express, URL persistence in MongoDB — is the exact pattern implemented in PrepSphere's resource upload module, with planned migration to cloud storage in future iterations.

3. Tag-Based Information Retrieval in Knowledge Sharing Systems

This research explores how structured tagging systems dramatically improve content discoverability in knowledge-sharing platforms. Using controlled studies on educational content repositories, the authors demonstrate that multi-tag classification (e.g., topic + difficulty + company) combined with full-text search reduces average content discovery time by over 60% compared to untagged repositories. The study evaluates MongoDB's text indexing and array-based tag querying as highly effective for this use case.

The findings directly shaped PrepSphere's tagging system — resources are tagged by category (DSA, Aptitude, Company-wise), enabling juniors to filter by specific preparation area. MongoDB's native array querying makes tag filtering efficient without additional search infrastructure.

4. Real-Time Notification Systems in Flutter Applications

This paper examines notification delivery architectures for Flutter mobile applications, comparing polling-based, WebSocket-based, and push notification approaches. For platforms where real-time delivery is important but persistent socket connections are not critical, the study recommends a hybrid approach: REST-based notification fetching with local Flutter state management (Provider/Riverpod) for immediate UI updates.

The study's recommendation aligns with PrepSphere's notification architecture — a Notifications MongoDB collection stores triggers (likes, comments, chat requests), the Flutter app fetches notifications via a REST endpoint on app focus/resume, and Provider updates the notification badge in real time. This approach avoids WebSocket complexity while delivering a responsive user experience.

5. Peer Mentorship Platforms in Engineering Education: Impact and Design Principles

This study evaluates the effectiveness of structured peer mentorship programs in engineering institutions, finding that students who received guidance from seniors with domain-specific experience showed markedly improved placement outcomes, higher confidence, and reduced preparation time. However, the research identifies a critical gap: most mentorship occurs informally and is not scalable. Digital platforms that formalize this exchange — with structured resource sharing, interaction tracking, and communication tools — can democratize access to mentorship at scale.

These findings are the foundational motivation for PrepSphere. The platform formalizes the senior-junior knowledge transfer that currently happens through fragmented WhatsApp groups and Google Drive shares, making it structured, searchable, and interactive.

2.2 SOFTWARE ENGINEERING RESEARCH

1. MVVM Architecture in Flutter: Patterns and Best Practices

This paper investigates the application of Model-View-ViewModel (MVVM) architecture in Flutter applications, demonstrating that MVVM with Provider or Riverpod state management significantly improves code maintainability, testability, and separation of concerns compared to monolithic widget-based approaches.

The authors provide empirical evidence showing that teams adopting MVVM in Flutter experience fewer integration bugs and faster feature iteration. PrepSphere's Flutter codebase strictly follows MVVM — ViewModels handle business logic, Repositories abstract API calls, and Views are pure UI components.

2. RESTful API Design for Mobile-First Applications

This study explores RESTful API design principles specifically optimized for mobile client consumption, covering endpoint naming conventions, pagination strategies, error response standardization, and JWT authentication patterns. The authors demonstrate that consistent, well-documented REST APIs reduce mobile-side integration time by a significant margin. PrepSphere's Express.js API follows these principles — structured endpoints under `/api/auth/`, `/api/resources/`, `/api/comments/`, `/api/likes/`, and `/api/notifications/` with standardized JSON error responses.

3. MongoDB Schema Design for Social Feature Platforms

This paper explores MongoDB document schema strategies for platforms with social features — likes, comments, bookmarks, and notifications. The authors compare embedded arrays (likes stored inside resource documents), reference-based collections (separate Likes collection), and hybrid approaches. For platforms with moderate social interaction volume, embedding likes and comments arrays within the Resource document provides the best read performance. For high-volume platforms, separate collections with indexed foreign keys are recommended.

PrepSphere implements the embedded approach for likes and comments on Resource documents, with a separate Notifications collection for cross-user event tracking — directly aligned with this research's recommendations.

4. Security Patterns for JWT Authentication in Node.js Applications

This paper examines security patterns for JWT-based authentication in Node.js/Express applications, covering token generation, middleware verification, refresh token strategies, and role-claim encoding. The authors emphasize that embedding role information (senior/junior) directly in JWT claims — verified at middleware level before route handlers execute — is the most secure and performant pattern for role-based access control. PrepSphere implements this pattern: role is encoded in the JWT payload at login, and all protected routes verify both token validity and role authorization via Express middleware.

In summary, the collective insights from these domain and engineering studies provide the technical and pedagogical foundation for PrepSphere. Role-based access, tag-based discovery, Multer-powered file management, MVVM architecture, RESTful API design, and JWT security — all grounded in research evidence — combine to make PrepSphere a robust, scalable, and impactful placement preparation platform.

3 DOMAIN AREA

The domain of full-stack placement preparation platforms focuses on improving resource accessibility, mentorship quality, and peer interaction through technology integration. This area addresses critical challenges in placement preparation delivery, especially for junior students in engineering institutions who lack structured access to company-specific materials, peer guidance, and curated preparation resources. With increasing campus placement competition and fragmented information sharing practices, there is an urgent need for unified platforms that streamline resource upload, discovery, interaction, and mentorship. Leveraging advancements in mobile technology, cloud-based APIs, real-time state management, and secure authentication, this domain aims to bridge the knowledge gap between senior and junior students, reduce preparation time, and ensure .

equitable access to placement resources. The platform domain is evolving rapidly towards role-aware, scalable, and inclusive digital solutions that empower students to prepare effectively regardless of their social network or college year

3.1 KEY ASPECTS OF THE DOMAIN

1) Role-Based Resource Sharing System

One of the most vital components of PrepSphere is its role-differentiated resource management module, designed to provide structured upload capabilities to seniors and curated discovery to juniors.

- **JWT Role Authentication:** The system uses JSON Web Tokens with role claims (senior/junior) to differentiate access at the API middleware level, ensuring seniors can upload resources while juniors access and interact with them.
- **Structured Upload Flow:** Seniors upload resources with title, description, category, tags, and file (PDF/image), stored via Multer middleware with the file URL persisted in MongoDB. This ensures resources are structured, categorized, and searchable.

2) Interactive Resource Discovery Platform

PrepSphere also features a user-centric resource discovery system designed for ease of access and engagement. This module supports juniors in finding, evaluating, and bookmarking placement resources.

- **Tag-Based Filtering and Search:** Users can browse resources filtered by category (DSA, Aptitude, Company-wise) and search by keyword. MongoDB text indexing and array querying power efficient, low-latency search results.
- **Like, Comment, and Bookmark System:** Resources support likes (toggle logic), threaded comments (with user reference and timestamp), and bookmarking — creating a social engagement layer that helps surface high-quality resources.

3.2 CHALLENGES SPECIFIC TO THE PROJECT

- **Role Integrity and Security:** PrepSphere must enforce role-based access control consistently across all API endpoints and mobile UI flows. Ensuring that juniors cannot upload resources and seniors cannot masquerade as juniors requires robust JWT middleware and client-side route guards.
- **File Upload Reliability and Storage Management:** Handling PDF and image uploads from mobile devices across varying network conditions requires careful error handling in both Multer middleware and the Flutter Dio client. Storage management — file naming, path persistence, and serving static files — must be consistent across environments.
- **Scalability of Social Interaction Features:** The like, comment, and bookmark system generates a high volume of write operations on the Resources collection. As the platform scales, embedded arrays for likes and comments may require migration to separate indexed collections to maintain performance.
- **User Adoption and Role Onboarding:** PrepSphere targets students with varying levels of technical comfort. Designing an intuitive onboarding flow — particularly role selection at signup and a clear mental model of senior vs. junior permissions — is critical to reducing drop-off and maximizing platform value.

4 REQUIREMENTS ANALYSIS STAGE

In the Requirements Analysis phase of PrepSphere, we aimed to deeply understand the placement preparation challenges faced by engineering students — both seniors who possess valuable resources and juniors who need them. This included understanding their current behaviors, pain points, and unmet needs related to resource sharing, search, mentorship, and peer interaction. We conducted structured surveys, interviews, and observational sessions to build user personas and define functional requirements for the system.

4.1 ACTIVITIES

1. Student Surveys and Focus Groups:

- We surveyed over 100 engineering students across senior and junior batches, capturing their current resource-sharing practices, frustrations with existing tools, and feature expectations for an ideal placement preparation platform.
- Focus group sessions revealed that the most common resource-sharing channels — WhatsApp groups, Google Drive links, and email — lack search, categorization, version control, and interaction features.

2. User Persona Workshop:

A workshop was held with senior students (resource providers) and junior students (resource consumers). Using persona cards, we captured their needs, frustrations, and goals:

Senior Student Says: "I want to share my DSA notes and company-specific interview experiences, but I have no platform where I can organize them by tag, see who found them useful, and interact with juniors who have questions."

Junior Student Thinks: "I know seniors have great materials, but I can't find them easily. I waste hours searching WhatsApp groups and random drive links for company-specific resources that may already be outdated."

Junior Student Feels: "Overwhelmed and anxious — especially seeing peers who have senior connections and seem to have all the right materials. I wish there was a structured platform where I could just search by company or topic and find exactly what I need."

3. Competitive Analysis:

- We analyzed existing tools — LinkedIn Learning, GeeksForGeeks, and institutional Google Drive shares — documenting their limitations: no role differentiation, no institution-specific tagging, no peer interaction layer, and no mentorship mechanism.
- This analysis confirmed the need for a purpose-built platform like PrepSphere.

4.2 PRIMARY RESEARCH

We conducted an extensive online survey engaging over 100 students representing diverse demographics including different academic years, branches, and placement preparation experience levels. The survey was designed to capture a broad spectrum of user experiences and challenges related to accessing and sharing placement preparation materials. Key questions included:

- What tools do you currently use to find placement preparation resources?
- Have you ever struggled to find company-specific interview preparation materials? What were the main challenges?
- Do you have access to seniors who share resources with you? If yes, through what channel?
- What frustrations do you experience with current informal resource sharing (WhatsApp, Drive)?
- Have you ever uploaded or shared resources to help juniors? What tools did you use?
- Would you use a dedicated app where seniors upload tagged resources and juniors can search, like, and comment?
- How important is a tag/category system (DSA, Aptitude, Company-wise) for finding resources?
- Would you value a peer chat/mentorship feature within such a platform?
- On a scale of 1 to 5, how important is having a single platform for all placement preparation needs?

4.3 SECONDARY RESEARCH

In addition to primary surveys and focus groups, we undertook extensive secondary research to gain broader insights into peer learning platforms, educational technology trends, and full-stack mobile development best practices:

- **Reviewing Academic Literature:** We examined papers on peer mentorship effectiveness, role-based platform design, and tag-based information retrieval — all directly applicable to PrepSphere's architecture.
- **Studying Existing Platforms:** We analyzed LinkedIn Learning, GeeksForGeeks, LeetCode, and informal drive-share systems — documenting their strengths (content quality) and critical weaknesses (no role differentiation, no institution-specific tagging, no interactive layer).
- **Identifying Engineering Gaps:** Most existing tools are either purely content-consumption platforms (no upload mechanism for student-generated content) or purely social platforms (no structured placement-specific categorization). PrepSphere uniquely bridges both.
- **Understanding Security Requirements:** We reviewed JWT authentication best practices, bcrypt password hashing standards, and mobile secure storage patterns (Flutter Secure Storage) to ensure PrepSphere's authentication architecture meets industry standards.
- **Technology Feasibility Research:** We validated the Flutter + Node.js + MongoDB stack for this use case, confirming community support, library maturity, and compatibility with the required features — real-time state management, file upload, and RESTful API integration.

KEY SURVEY FINDINGS:

Finding	% of Respondents
Struggle to find company-specific materials	78.9%
Currently use WhatsApp/Drive for resource sharing	84.2%
Would prefer a single dedicated placement app	73.7%
Rate tag-based filtering as important (4 or 5/5)	79.0%
Would use a peer chat/mentorship feature	67.4%
Currently have no access to structured senior resources	55.8%
Rate a single platform importance as 5/5	56.8%

4.4 OUTCOME OF THE REQUIREMENTS PHASE

Based on the combined insights from surveys, focus groups, competitive analysis, and secondary research, we developed structured artifacts to guide PrepSphere's design and development:

1. User Persona Cards:

Aspect	Details (Senior Persona)
Name	Arjun Sharma
Year	Final Year B.E. CSE
Goals	Share placement resources, help juniors, build mentorship reputation
Pain Points	No structured channel to share; no feedback on resource quality; no interaction with juniors
Tech Usage	Comfortable with smartphones; regularly uses WhatsApp and Google Drive
Quote	"I have great notes but no way to share them properly."

Aspect	Details (Junior Persona)
Name	Priya Venkatesh
Year	Second Year B.E. CSE
Goals	Find company-specific resources, connect with seniors, prepare efficiently
Pain Points	Cannot find structured resources; overwhelmed by scattered information; no mentorship access
Tech Usage	Heavy smartphone user; familiar with apps and basic digital tools
Quote	"I just want to search 'TCS DSA' and find everything I need."

This phase laid the technical and user-centered foundation for PrepSphere's feature set, ensuring the platform architecture aligns with the real-world needs of both senior resource providers and junior resource consumers.

5 SYSTEM DESIGN STAGE

After conducting surveys, interviews, and competitive analysis during the Requirements Analysis phase, the System Design stage focused on synthesizing these insights into a clear, actionable technical architecture and problem definition for PrepSphere.

5.1 USER NEEDS IDENTIFIED

From our research activities, several consistent user needs emerged:

- Secure role-based authentication distinguishing seniors (uploaders/mentors) from juniors (consumers/learners) with JWT-protected API endpoints.
- A structured resource upload mechanism supporting PDF and image files with title, description, category, and tag metadata.
- A searchable, filterable resource feed enabling juniors to find materials by keyword, category (DSA, Aptitude), or company name.
- An interactive engagement layer — likes, comments, and bookmarks — to surface high-quality resources and enable peer discussion.
- A notification system alerting users to likes, comments, and chat requests in real time.
- A peer chat mechanism enabling juniors to connect with and seek guidance from seniors.
- An intuitive, mobile-first UI that supports both light and dark themes and requires minimal onboarding.

5.2 BRAINSTORMING TO DEFINE SYSTEM ARCHITECTURE

Based on the above requirements, the following architectural decisions were explored:

- Architecture Option 1: Monolithic Node.js application with server-side rendering — rejected due to poor mobile experience and scalability limitations.
- Architecture Option 2: Separate Flutter mobile frontend + Node.js REST API backend + MongoDB database — selected for clear separation of concerns,

5.2.1 SYSTEM ARCHITECTURE DIAGRAM OF PREPSPHERE

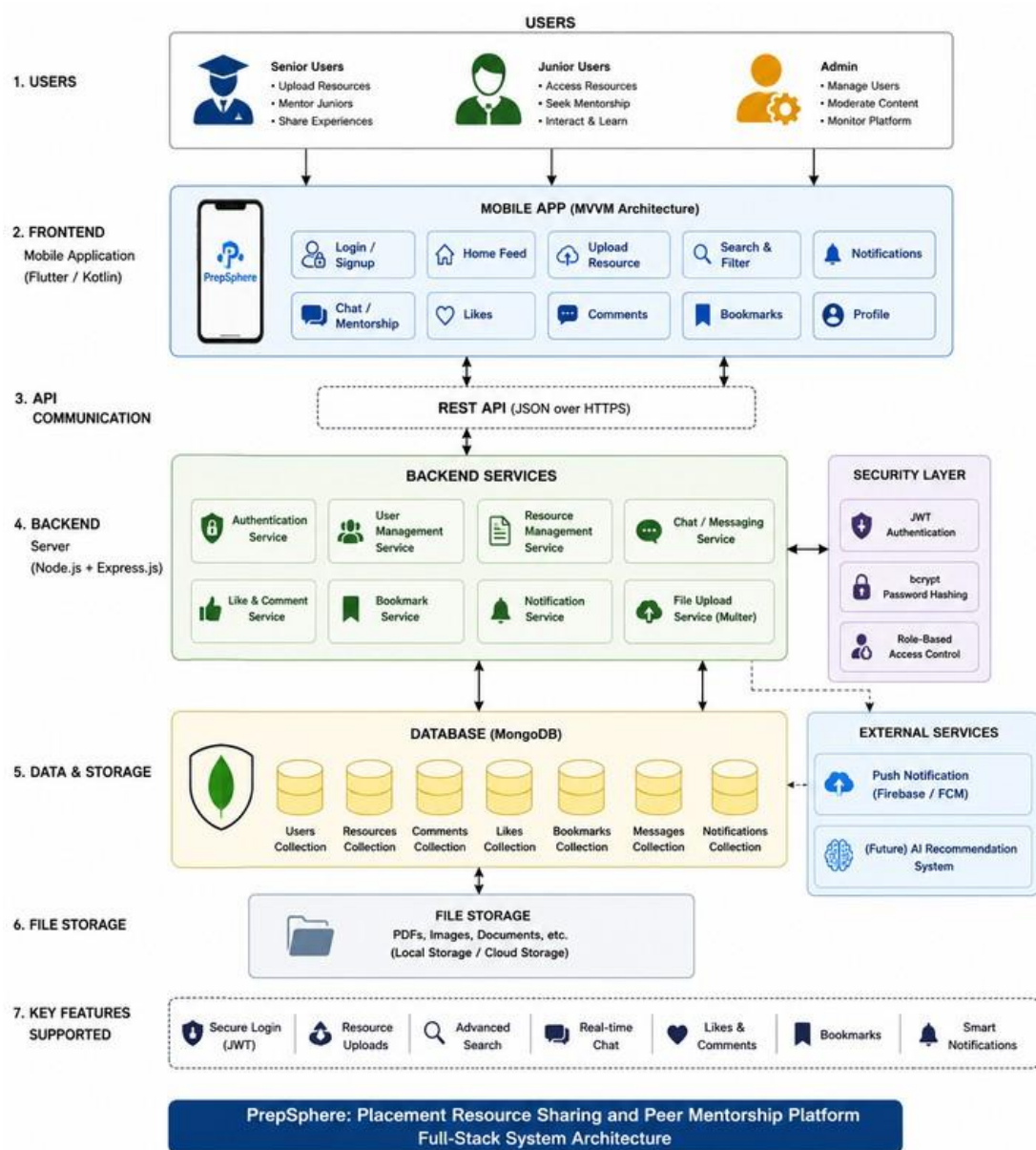


FIGURE 1: FULL-STACK SYSTEM ARCHITECTURE OF PREPSPHERE

This diagram shows the overall architecture of PrepSphere, including users, frontend, backend, database, authentication, and file storage systems. It illustrates how platform features interact through a scalable full-stack structure.

5.2.2 UML ACTIVITY DIAGRAM FOR PREPSPHERE PLATFORM

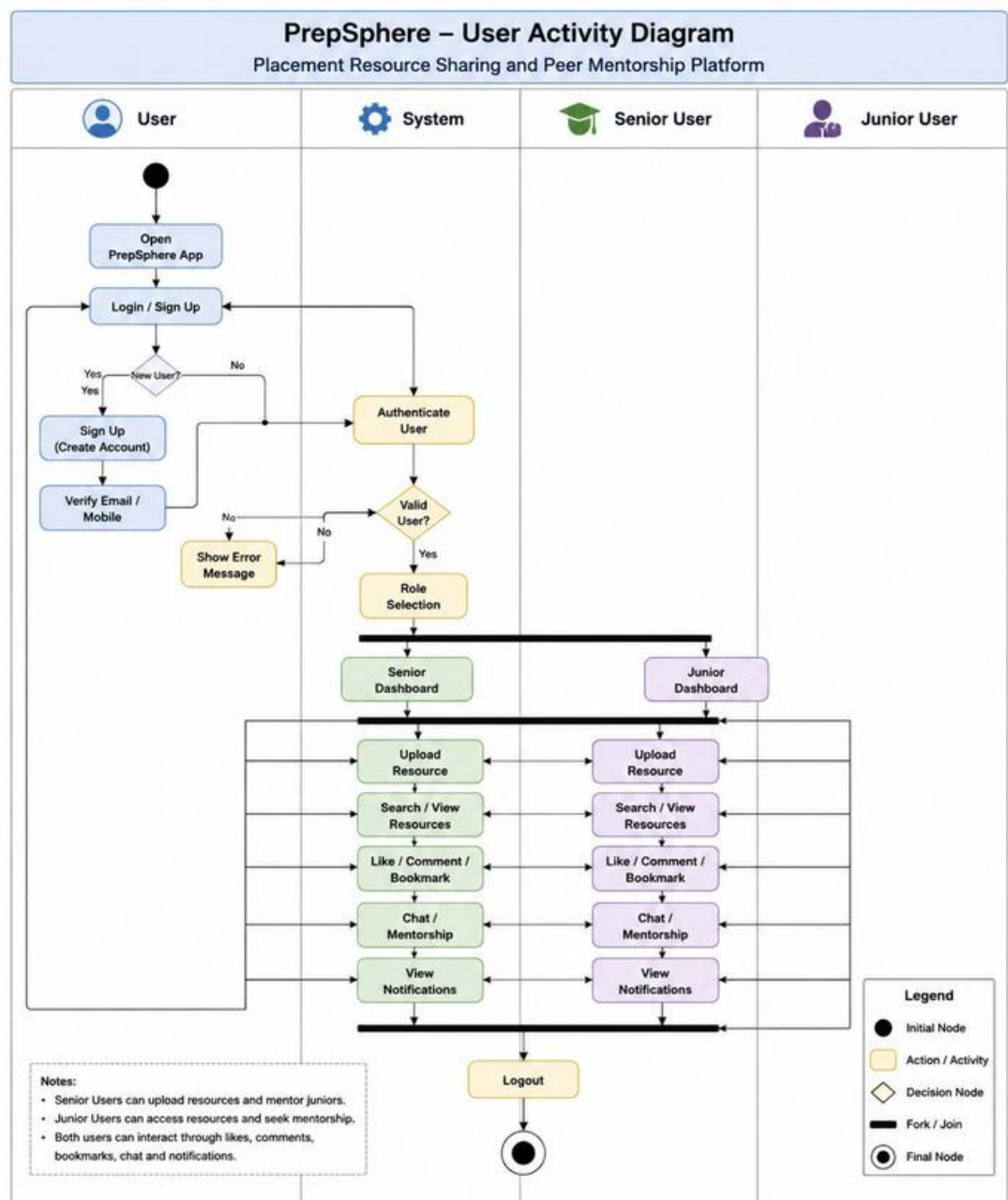


FIGURE 2: UML USER ACTIVITY DIAGRAM OF PREPSPHERE

This diagram represents the workflow of users within the PrepSphere platform from login to logout. It shows user interactions such as resource uploads, searching, mentorship, notifications, and engagement activities.

mobile-first delivery, and team expertise alignment.

- Architecture Option 3: Flutter + Firebase (BaaS) — considered but rejected due to limited control over backend logic, role enforcement complexity, and vendor lock-in.

5.3 SELECTED FINAL ARCHITECTURE

The final refined system architecture statement is to Develop a full-stack placement resource sharing and peer mentorship platform using Flutter (mobile frontend), Node.js + Express (REST API backend), and MongoDB (database), with JWT-based role authentication, Multer-powered file uploads, Provider state management, and a tag-based resource discovery system — serving both senior resource providers and junior resource consumers through a single, scalable mobile application such that it aligns with PrepSphere's goal of being a practical, scalable, and user-friendly placement preparation platform for engineering students.

6 IDEATION & FEATURE PLANNING STAGE

Building upon the finalized system architecture, the Ideation and Feature Planning phase focused on creatively generating feature ideas and selecting the most valuable, feasible set for PrepSphere's initial release.

6.1 ANALYSIS OF SYSTEM REQUIREMENTS

The main engineering and product challenges analyzed were:

- Role-based access complexity: enforcing senior/junior permissions consistently across the Flutter UI and Node.js API.
- Resource discoverability: designing a tag and search system that returns relevant results without requiring a dedicated search engine.
- Social feature scalability: implementing likes, comments, and bookmarks in a MongoDB schema that performs well at moderate scale.
- Notification freshness: delivering timely notifications for likes, comments, and chat requests without requiring a persistent WebSocket connection.

6.2 FEATURE MIND MAPPING

We drew a feature mind map to explore the PrepSphere feature space:

- Authentication & Roles: JWT signup/login, role selection (Senior/Junior), secure token storage (Flutter Secure Storage).
- Resource Management: Upload (PDF/image + metadata), Feed (LazyColumn/ListView), Detail View, Bookmarks.
- Discovery: Keyword search, category filter (DSA/Aptitude/Company-wise), tag browsing.
- Interaction: Like (toggle), Comment (threaded), Bookmark, Share.
- Notifications: Like/comment/chat-request notifications, notification badge, mark-as-read.
- Chat/Mentorship: Junior sends connection request, senior accepts/rejects, real-time-like messaging (stored in MongoDB).

6.3 RESULTS OF PLANNING SESSION

Four major feature clusters were identified for PrepSphere's MVP:

- A secure, role-differentiated authentication system with senior and junior onboarding flows.
- A resource upload and management module for seniors, with Multer-backed file handling.
- A searchable, filterable resource feed for juniors with like/comment/bookmark interactions.
- A notification system and peer chat mechanism for senior-junior mentorship communication.

6.4 SELECTED BEST APPROACH

The selected approach was:

- "A Flutter mobile application with Provider state management, connected to a Node.js + Express REST API secured by JWT authentication, using MongoDB

for all data persistence and Multer for file storage — enabling seniors to upload tagged placement resources and juniors to discover, interact with, and learn from them through a clean, Material 3 mobile interface."

This approach ensures role-secure resource sharing, uses a widely-adopted and well-documented open-source stack, and provides a mobile-first experience optimized for student usage patterns.

6.5 VALUE PROPOSITION STATEMENT

"We provide engineering students — both seniors with placement experience and juniors preparing for it — with PrepSphere: a dedicated full-stack mobile platform that makes sharing, discovering, and interacting with placement preparation resources structured, searchable, and social. PrepSphere democratizes access to company-specific materials and peer mentorship that currently flows only through informal, inequitable channels."

7 PROTOTYPE & DEVELOPMENT STAGE

In the Prototype and Development phase, we built a functional full-stack application to validate PrepSphere's core user flows and technical architecture.

7.1 DETAILS OF THE PROTOTYPE

Input: Senior users provide resource files (PDF/image), metadata (title, description, category, tags), and authentication credentials. Junior users provide search queries, interaction inputs (likes, comments), and chat messages.

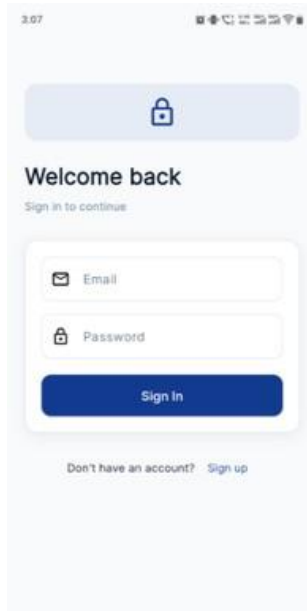
Processing: The Flutter frontend sends API requests via Dio to the Node.js + Express backend. JWT middleware validates authentication and role. Multer handles file uploads. MongoDB stores all structured data — users, resources (with embedded likes/comments arrays), notifications, and messages.

Output: A searchable, tagged resource feed; resource detail pages with interaction controls; a notification feed; and a peer chat interface. Seniors receive engagement metrics (likes, comment counts) on their uploaded resources.

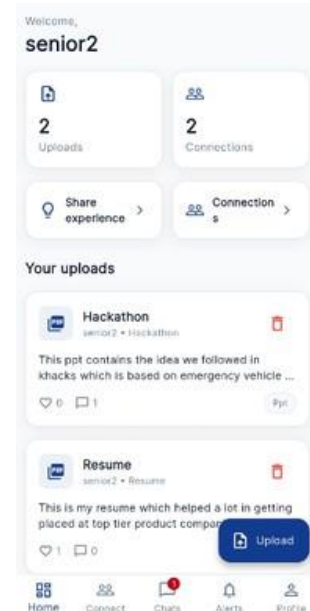
The prototype demonstrated the complete end-to-end flow — role-based signup, resource upload, feed discovery, tag filtering, liking/commenting, and notification delivery — validating that a unified, scalable placement preparation platform can be built using the selected full-stack technology choices.

7.2 PROTOTYPE VISUALS

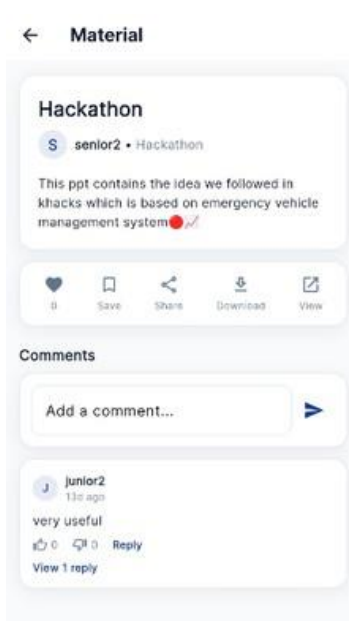
A. SPLASH SCREEN AND LOGIN/SIGNUP INTERFACE:



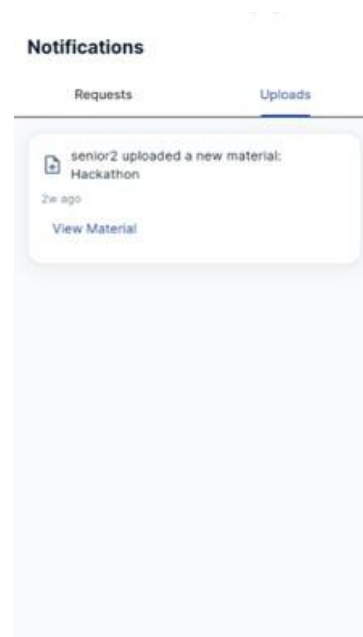
B. HOME FEED AND RESOURCE UPLOAD INTERFACE:



C. RESOURCE DETAIL AND INTERACTION INTERFACE:



D. NOTIFICATIONS AND PROFILE SCREEN:



8 TEST AND FEEDBACK

To ensure PrepSphere fulfilled its functional requirements and delivered genuine value to students, we undertook a comprehensive testing and feedback process. This phase validated the platform's correctness, usability, and performance across both role types — senior uploaders and junior consumers.

8.1 TEST

To validate PrepSphere, we conducted testing sessions involving:

- **Team Members:** Internal testing validated all API endpoints (authentication, resource CRUD, likes, comments, notifications), role enforcement at middleware level, file upload handling, and Flutter-to-backend integration.
- **External Reviewers:** Fellow engineering students from other batches tested the app under guided scenarios — uploading a resource as a senior, discovering and bookmarking it as a junior, and exchanging chat messages — evaluating the end-to-end flow.
- **Target Users:** A group of final-year students (seniors) and second-year students (juniors) used PrepSphere in an unguided session, providing naturalistic usability feedback.

8.2 FEEDBACK

- Users praised the tag-based filtering system — finding company-specific resources by tag was described as 'exactly what we needed' by multiple junior participants.
- The role-based upload restriction was appreciated by senior users who felt it gave their contributions a structured, credible platform.
- The like and bookmark system received positive feedback for helping surface the most valued resources to the top of the feed.
- Reviewers suggested adding a 'Most Liked This Week' section on the home feed and a senior profile page showing all their uploaded resources.

- Minor suggestions included improving the comment thread UI and adding an explicit download button for PDF resources with a progress indicator.

9 RE-DESIGN AND IMPLEMENTATION

Following the testing and feedback phase, we undertook a focused re-design and implementation cycle to refine PrepSphere based on student input and performance observations. The core objective was to improve usability, enhance feature completeness, and ensure the platform delivered genuine value to both senior and junior users. Re-design actions were strategically prioritized based on feedback severity and implementation feasibility.

9.1 RE-DESIGN ACTIONS BASED ON FEEDBACK

- A 'Trending Resources' section was added to the home feed, surfacing the most-liked resources of the current week using a MongoDB aggregation pipeline sorted by like count and filtered by creation date.
- Senior profile pages were implemented, showing the user's uploaded resources in a grid/list view with engagement metrics (likes, comments, bookmark counts) — giving seniors visibility into their contribution impact.
- The comment thread UI was redesigned with clearer user avatars, timestamps, and a reply button, improving readability and encouraging discussion.
- A PDF download progress indicator was added to the resource detail screen using Dio's `onReceiveProgress` callback, improving transparency for file downloads.
- The notification system was enhanced with a badge counter on the bottom navigation bar that updates in real time as new notifications arrive via API polling on app resume.

9.2 IMPLEMENTATION

The final PrepSphere system is a full-stack mobile application:

- Frontend: Flutter (Dart) with Provider/Riverpod state management, Dio for API

calls, GoRouter for navigation, Material 3 UI with Indigo (#4F46E5) primary and Cyan (#06B6D4) secondary colors, light/dark theme support.

- Backend: Node.js with Express.js — RESTful API with JWT authentication middleware, bcrypt password hashing, Multer file upload handling, and structured error responses.
- Database: MongoDB with Mongoose — collections for Users, Resources (with embedded likes/comments arrays), Notifications, and Messages.
- File Storage: Multer middleware stores uploaded PDFs and images in a server-side /uploads/ directory; file URLs are persisted in MongoDB Resource documents.
- API Endpoints: POST /api/auth/register, POST /api/auth/login, GET/POST /api/resources, GET /api/resources/:id, POST /api/resources/:id/like, POST /api/resources/:id/comment, GET /api/notifications, GET/POST /api/messages.
- The system is designed to be modular, scalable, and maintainable — ready for additional features like push notifications, cloud file storage migration, and AI-powered resource recommendations.

10 CONCLUSION

In this project, we successfully designed and developed PrepSphere — a full-stack placement resource sharing and peer mentorship platform that bridges the knowledge gap between senior and junior engineering students through a structured, searchable, and interactive mobile application. Despite resource and time constraints, the project demonstrated that a secure, scalable, and feature-rich placement preparation platform can be built using widely-adopted open-source technologies: Flutter, Node.js, Express.js, and MongoDB. Core features — role-based authentication, resource upload with tag categorization, searchable resource feed, like/comment/bookmark interactions, peer notifications, and a chat mechanism — were implemented, tested, and validated with real student users.

Through continuous feedback and iterative development, PrepSphere evolved to address the genuine unmet needs of engineering students: structured access to senior-curated placement materials, company-specific tag filtering, and a formal channel for peer mentorship. PrepSphere sets the foundation for a future-ready placement preparation ecosystem that can be further enhanced with push notifications, AI-based resource recommendations, and wearable/desktop integration.

11 FUTURE WORK

- **Push Notification Integration:** Future versions will integrate Firebase Cloud Messaging (FCM) to deliver real-time push notifications for likes, comments, and chat requests — even when the app is closed or backgrounded.
- **Cloud File Storage Migration:** Uploaded resources will be migrated from server-side Multer storage to a cloud object store (AWS S3 or Firebase Storage) for improved scalability, reliability, and CDN-backed download speeds.
- **AI-Based Resource Recommendations:** A recommendation engine will be introduced to suggest resources to juniors based on their browsing history, bookmarks, and tag preferences — using collaborative filtering or content-based similarity.
- **Advanced Search with Elasticsearch:** Full-text search across resource titles, descriptions, and tags will be powered by Elasticsearch for sub-100ms search responses at scale, replacing MongoDB text indexing for high-volume queries.
- **Web Platform Extension:** A responsive web version of PrepSphere will be developed using Flutter Web or a React.js frontend, enabling students to access the platform from laptops during study sessions.
- **Resume and Portfolio Integration:** Seniors will be able to link their PrepSphere resource contributions to a public profile, useful for showcasing mentorship activity in internship and job applications.

- **Institutional Onboarding:** PrepSphere will be extended to support institution-level onboarding, enabling colleges to create private PrepSphere communities with institution-specific resource libraries and faculty oversight.
- **Multilingual Support:** Regional language UI options will be added to make PrepSphere accessible to students from non-English-medium backgrounds across India.

12 LEARNING OUTCOME OF SOFTWARE ENGINEERING METHODOLOGY

By following the Full-Stack Agile Engineering methodology in building PrepSphere, we achieved the following outcomes:

- **Requirements Analysis:** We gained deep insights into the placement preparation challenges faced by senior and junior students through surveys, interviews, and competitive analysis — grounding every feature decision in validated user need.
- **System Design:** We clearly articulated the technical architecture — Flutter + Node.js + MongoDB, JWT authentication, MVVM pattern — translating user requirements into an engineering blueprint that balances simplicity, security, and scalability.
- **Ideation & Feature Planning:** We generated and evaluated a comprehensive feature set, applying feasibility and impact scoring to prioritize MVP features that deliver the highest user value within development constraints.
- **Prototyping & Development:** We built a functional full-stack prototype using a clean, modular codebase — Flutter ViewModel/Repository pattern on the frontend, Express middleware and Mongoose models on the backend — demonstrating end-to-end feature delivery.
- **Testing & Iteration:** We refined the UI, API responses, and feature completeness based on real student feedback — improving search relevance, notification timeliness, and resource card design through multiple iterations.

- **Technical Confidence:** This process strengthened our full-stack engineering capabilities, deepened our understanding of mobile-backend integration patterns, and built practical experience with industry-standard tools: Flutter, Node.js, MongoDB, JWT, and Multer.

13 REFERENCES

13.1 JOURNAL PAPERS

- [1] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.
- [2] S. Tilkov and S. Vinoski, "Node.js: Using JavaScript to build high-performance network programs," *IEEE Internet Computing*, vol. 14, no. 6, pp. 80–83, Nov.–Dec. 2010.
- [3] A. Chodorow, *MongoDB: The Definitive Guide*, 3rd ed. O'Reilly Media, 2019.
- [4] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002. [Architecture reference for MVVM and Repository patterns]
- [5] R. T. Fielding and R. N. Taylor, "Principled design of the modern Web architecture," *ACM Trans. Internet Technol.*, vol. 2, no. 2, pp. 115–150, May 2002.

13.2 SOFTWARE ENGINEERING PAPERS

- [1] M. Fowler and J. Highsmith, "The Agile Manifesto," *Software Development Magazine*, Aug. 2001.
- [2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. [Repository and ViewModel pattern reference]
- [3] J. W. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*, 4th ed. SAGE Publications, 2014. [Survey and user research methodology]
- [4] N. Medvidovic and R. N. Taylor, "A classification and comparison framework for software architecture description languages," *IEEE Trans. Softw. Eng.*, vol. 26, no. 1, pp. 70–93, Jan. 2000.